

Nguyễn Minh Nhật B2205896

Lab4. Stored subprograms and triggers

Part 2:

1. Write a code in PL/SQL to create a trigger that automatically updates a 'last_modified' timestamp whenever a row in a specific table is updated for the following table:

--Code

```
CREATE TABLE employee (  
    employee_id NUMBER PRIMARY KEY,  
    first_name VARCHAR2(50),  
    last_name VARCHAR2(50),  
    last_modified TIMESTAMP  
);
```

--Create trigger

```
CREATE OR REPLACE TRIGGER update_last_modified_trg  
BEFORE UPDATE ON employee  
FOR EACH ROW  
BEGIN  
    :NEW.last_modified := SYSDATE;  
END;
```

nhut.sql

Worksheet Query Builder

```

CREATE TABLE employee (
  employee_id NUMBER PRIMARY KEY,
  first_name VARCHAR2(50),
  last_name VARCHAR2(50),
  last_modified TIMESTAMP
);

CREATE OR REPLACE TRIGGER update_last_modified_trg
BEFORE UPDATE ON employee
FOR EACH ROW
BEGIN
  :NEW.last_modified := SYSDATE;
END;

```

Script Output x

Task completed in 0.024 seconds

Table EMPLOYEE created.

Trigger UPDATE_LAST_MODIFIED_TRG compiled

-- Test

nhut.sql

Worksheet Query Builder

```

UPDATE employee
SET first_name = 'Nhut'
WHERE employee_id = 1;

SELECT * FROM employee;

```

Script Output x Query Result x

All Rows Fetched: 2 in 0.006 seconds

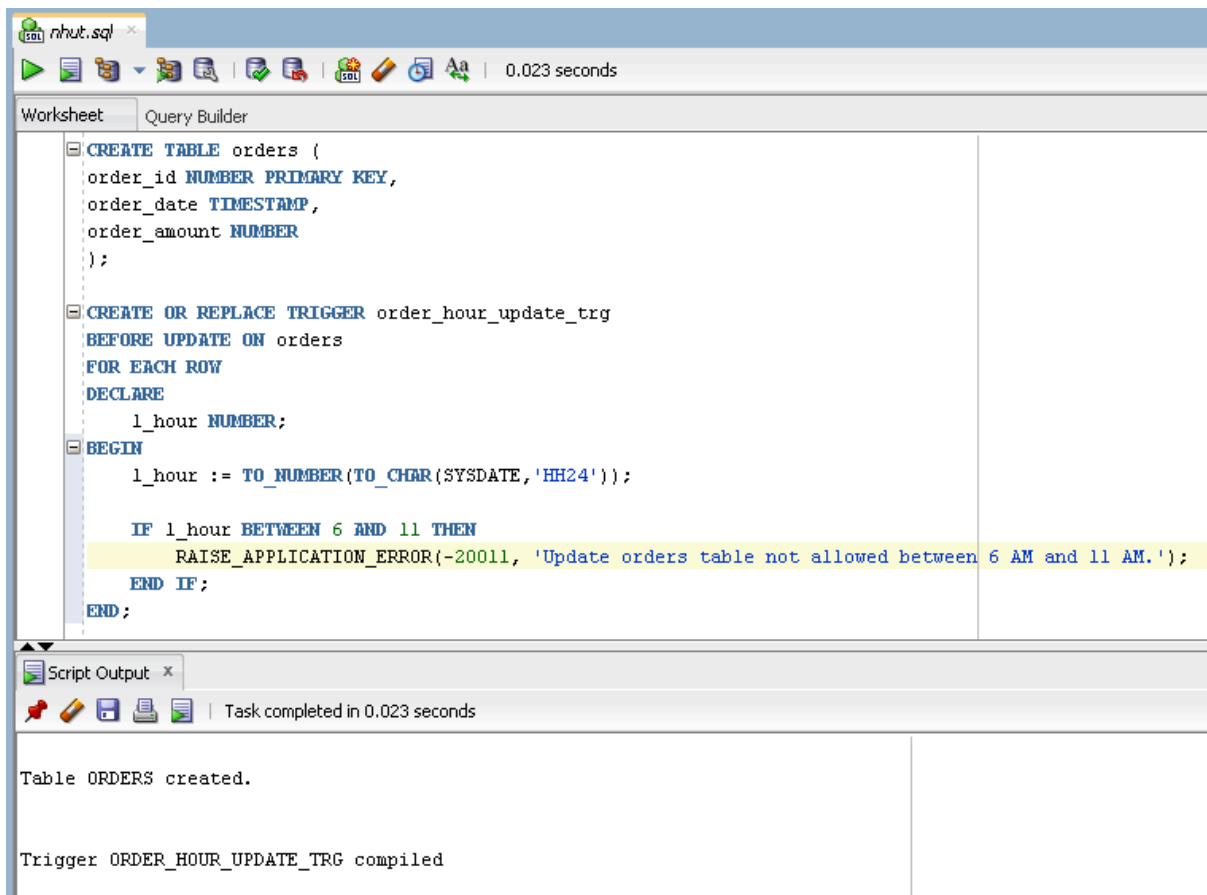
	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	LAST_MODIFIED
1	1	Nhut	Doe	01-NOV-24 08.38.30.000000000 AM
2	2	Jane	Smith	01-NOV-24 08.38.01.905000000 AM

2. Write a code in PL/SQL to create a trigger that prevents updates on a certain column during specific hours of the day for the following table:

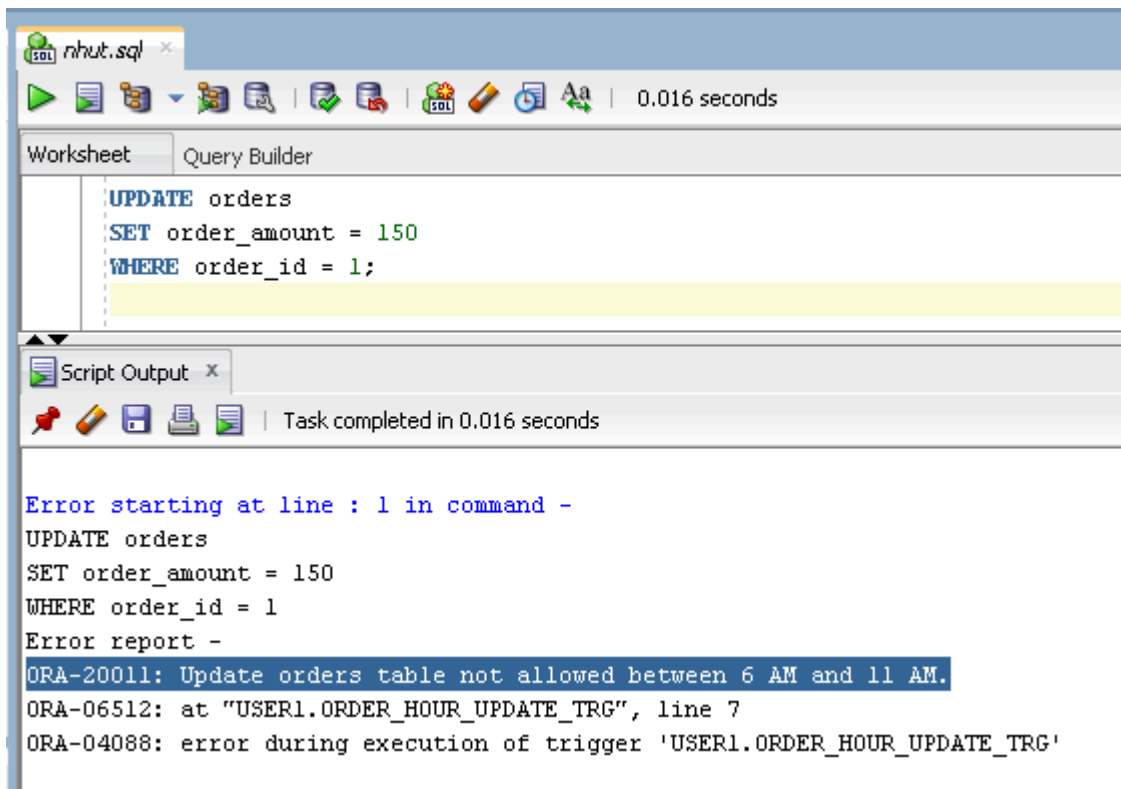
--Code

```
CREATE TABLE orders (  
    order_id NUMBER PRIMARY KEY,  
    order_date TIMESTAMP,  
    order_amount NUMBER  
);  
  
--Create trigger  
CREATE OR REPLACE TRIGGER order_hour_update_trg  
BEFORE UPDATE ON orders  
FOR EACH ROW  
DECLARE  
    l_hour NUMBER;  
BEGIN  
    l_hour := TO_NUMBER(TO_CHAR(SYSDATE,'HH24'));  
  
    IF l_hour BETWEEN 6 AND 11 THEN  
        RAISE_APPLICATION_ERROR(-20011, 'Update orders table not allowed between 6  
AM and 11 AM.');
```

```
END IF;  
END;
```



--Test



3. Write a code in PL/SQL to implement a trigger that maintains a transaction history log whenever a row is deleted from the following table:

--Code

```
CREATE TABLE employees (  
    employee_id NUMBER PRIMARY KEY,  
    first_name VARCHAR2(50),  
    last_name VARCHAR2(50)  
);
```

```
CREATE TABLE employees_log (  
    employee_id NUMBER,  
    first_name VARCHAR2(50),  
    last_name VARCHAR2(50),  
    time_delete TIMESTAMP DEFAULT SYSTIMESTAMP  
);
```

--Create trigger

```
CREATE OR REPLACE TRIGGER trg_log_employee_deletion  
AFTER DELETE ON employees  
FOR EACH ROW  
BEGIN  
    INSERT INTO employees_log(employee_id, first_name, last_name, time_delete)  
    VALUES (:OLD.employee_id, :OLD.first_name, :OLD.last_name, SYSTIMESTAMP);
```

END;

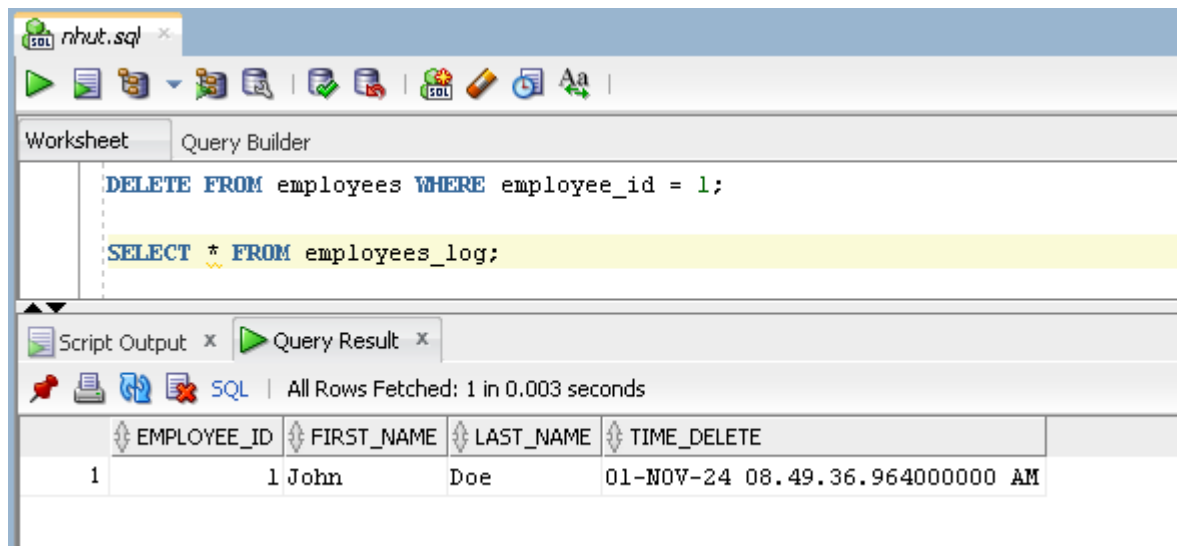
The screenshot shows the SQL Developer interface with a file named 'nhut.sql'. The 'Query Builder' tab is active, displaying the following SQL script:

```
CREATE TABLE employees (  
  employee_id NUMBER PRIMARY KEY,  
  first_name VARCHAR2(50),  
  last_name VARCHAR2(50)  
);  
  
CREATE TABLE employees_log (  
  employee_id NUMBER,  
  first_name VARCHAR2(50),  
  last_name VARCHAR2(50),  
  time_delete TIMESTAMP DEFAULT SYSTIMESTAMP  
);  
  
CREATE OR REPLACE TRIGGER employees_log_trg  
AFTER DELETE ON employees  
FOR EACH ROW  
BEGIN  
  INSERT INTO employees_log(employee_id, first_name, last_name, time_delete)  
  VALUES (:OLD.employee_id, :OLD.first_name, :OLD.last_name, SYSTIMESTAMP);  
END;
```

Below the script, the 'Script Output' and 'Query Result' tabs are visible. The status bar indicates 'Task completed in 0.042 seconds'. The output area shows the following messages:

```
Table EMPLOYEES created.  
  
Table EMPLOYEES_LOG created.  
  
Trigger EMPLOYEES_LOG_TRG compiled
```

-- Test



4. Write a code in PL/SQL to develop a trigger that enforces referential integrity by preventing the deletion of a parent record if child records exist for the following table:

--Code

```
CREATE TABLE departments (
    department_id NUMBER PRIMARY KEY,
    department_name VARCHAR2(50)
);

CREATE TABLE employees (
    employee_id NUMBER PRIMARY KEY,
    first_name VARCHAR2(50),
    last_name VARCHAR2(50),
    department_id NUMBER,
    CONSTRAINT fk_dept FOREIGN KEY (department_id) REFERENCES
departments (department_id)
);
```

--Create trigger

```
CREATE OR REPLACE TRIGGER prevent_deletion_parent_trg
BEFORE DELETE ON departments
FOR EACH ROW
DECLARE
    l_employee_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO l_employee_count
    FROM employees
    WHERE department_id = :OLD.department_id;

    IF l_employee_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20012, 'Cannot delete department which exist in
employees.');
```

```
END IF;  
END;
```

The screenshot displays the SQL Developer interface with a script named 'nhut.sql'. The script contains three SQL statements: creating a 'departments' table, creating an 'employees' table with a foreign key to 'departments', and creating a trigger 'prevent_deletion_parent_trg' that prevents deleting a department if it has associated employees. The 'Script Output' pane at the bottom shows the successful execution of these statements.

```
-- nhut.sql  
  
CREATE TABLE departments (  
    department_id NUMBER PRIMARY KEY,  
    department_name VARCHAR2(50)  
);  
  
CREATE TABLE employees (  
    employee_id NUMBER PRIMARY KEY,  
    first_name VARCHAR2(50),  
    last_name VARCHAR2(50),  
    department_id NUMBER,  
    CONSTRAINT fk_dept FOREIGN KEY (department_id) REFERENCES departments (department_id)  
);  
  
CREATE OR REPLACE TRIGGER prevent_deletion_parent_trg  
BEFORE DELETE ON departments  
FOR EACH ROW  
DECLARE  
    l_employee_count NUMBER;  
BEGIN  
    SELECT COUNT(*) INTO l_employee_count  
    FROM employees  
    WHERE department_id = :OLD.department_id;  
  
    IF l_employee_count > 0 THEN  
        RAISE_APPLICATION_ERROR(-20012, 'Cannot delete department which exist in employees.');    END IF;  
END;
```

Script Output x Query Result x

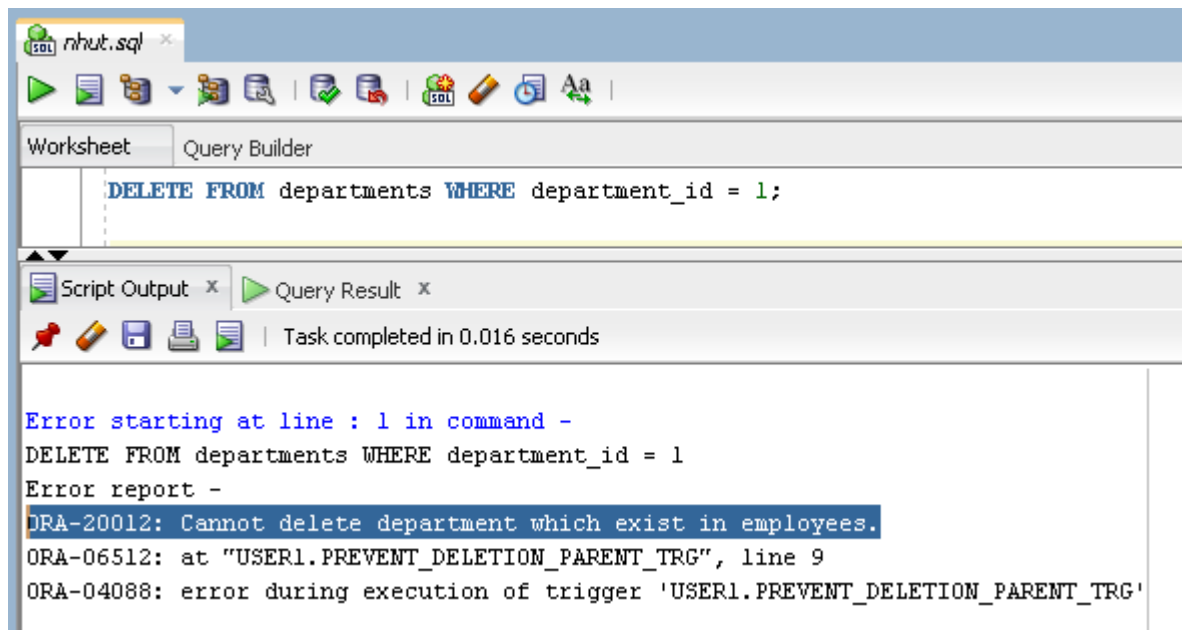
Task completed in 0.024 seconds

Table DEPARTMENTS created.

Table EMPLOYEES created.

Trigger PREVENT_DELETION_PARENT_TRG compiled

--Test



5. Write a code in PL/SQL to create a trigger that checks for duplicate values in a specific column and raises an exception if found for the following table:

--Code

```
CREATE TABLE products (
    product_id NUMBER PRIMARY KEY,
    product_name VARCHAR2(50)
);
```

--Create trigger

```
CREATE OR REPLACE TRIGGER check_duplicate_product_name_trg
BEFORE INSERT OR UPDATE OF product_name ON products
FOR EACH ROW
DECLARE
    l_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO l_count
    FROM products
    WHERE product_name = :NEW.product_name AND product_id != :NEW.product_id;

    IF l_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20013, 'Duplicate product name in a specific
column');
    END IF;
END;
```


The screenshot shows the SQL Developer interface with a worksheet titled 'nhut.sql'. The 'Query Builder' tab is active. The SQL code in the editor is as follows:

```
CREATE TABLE products (  
    product_id NUMBER PRIMARY KEY,  
    product_name VARCHAR2(50)  
);  
  
CREATE OR REPLACE TRIGGER dup_product_name_trg  
BEFORE INSERT OR UPDATE OF product_name ON products  
FOR EACH ROW  
DECLARE  
    l_count NUMBER;  
BEGIN  
    SELECT COUNT(*) INTO l_count  
    FROM products  
    WHERE product_name = :NEW.product_name AND product_id != :NEW.product_id;  
  
    IF l_count > 0 THEN  
        RAISE_APPLICATION_ERROR(-20013, 'Duplicate product name in a specific column');  
    END IF;  
END;
```

Below the editor, the 'Script Output' window shows the execution results:

```
Table PRODUCTS created.  
  
Trigger DUP_PRODUCT_NAME_TRG compiled
```

-- Test

The screenshot shows the SQL Developer interface with the same worksheet 'nhut.sql'. The 'Query Builder' tab is active. The SQL code in the editor is:

```
INSERT INTO products (product_id, product_name) VALUES (1, 'Widget');  
INSERT INTO products (product_id, product_name) VALUES (2, 'Widget');
```

The 'Script Output' window shows the execution results:

```
1 row inserted.  
  
Error starting at line : 2 in command -  
INSERT INTO products (product_id, product_name) VALUES (2, 'Widget')  
Error report -  
ORA-20013: Duplicate product name in a specific column  
ORA-06512: at "USER1.DUP_PRODUCT_NAME_TRG", line 9  
ORA-04088: error during execution of trigger 'USER1.DUP_PRODUCT_NAME_TRG'
```

— END —